FINSIGHT AI

TECHNICAL DESIGN & IMPLEMENTATION PLAN

Architecture baseline, engineering standards, delivery sequencing and release controls

FOUR-SPRINT PROJECT BASELINE | 18-21 OCTOBER 2026

Prepared from the supplied SRS, 72-entry Jira backlog, four-sprint allocation, 158-core-table schema reference and Sprint 1 test plan.

Scope baseline	72 Jira story/task entries across 11 epics
Schedule baseline	Exactly four sprints; do not use the superseded seven-day sequence
Technology baseline	Next.js/React • FastAPI • PostgreSQL + pgvector • Ollama/provider adapters
Source of truth	The SRS and SQL/schema package remain authoritative
Release guardrail	Paper trading and broker execution remain Release 2 only

Document control and engineering intent

Field	Value
Document type	Technical design and implementation plan
Version	1.0 - baseline for implementation
Prepared on	18 October 2026
Audience	Product owner, technical lead, backend/frontend engineers, data/ML engineers, QA, DevOps/security and reviewers
Primary use	Translate the approved requirements and the 72-entry Jira catalog into implementable architecture, contracts, data flows, sprint work and acceptance gates
Status	Proposed technical baseline. Validate implementation choices against the live repository and environment before treating them as deployed facts.

Executive implementation principle

Build a modular FastAPI application with explicit domain boundaries, PostgreSQL as structured source of truth, pgvector in that PostgreSQL deployment, provider/model adapters, and asynchronous workers for long-running operations. Avoid premature microservices; split deployable services only when scale, security isolation or team ownership justifies it.

Contents / navigation

- 1. Executive technical decisions and design summary
- 2. Source baseline, scope and release boundaries
- 3. Product domains and architecture requirements
- 4. Logical system architecture
- 5. Module boundaries and repository structure
- 6. Core application flows
- 7. API and service contract design
- 8. Data architecture and schema strategy
- 9. Market-data ingestion and data quality
- 10. Document processing, RAG and AI safety
- 11. Financial calculation correctness and reproducibility
- 12. Security, privacy and threat controls
- 13. Background jobs, reliability and observability
- 14. Testing strategy and release gates
- 15. Four-sprint implementation plan based on all 72 Jira entries
- 16. Cross-sprint dependency graph and parallel workstreams
- 17. CI/CD, deployment and operational runbooks
- 18. Non-functional target matrix
- 19. Architecture decisions, risks and open questions
- 20. Definition of Done and release acceptance
- Appendix A. Complete story-to-sprint allocation
- Appendix B. Source references and traceability map

1. Executive technical decisions and design summary

FinSight AI is a source-backed financial research workspace. It combines historical market and fundamental data, document ingestion and retrieval, conversational AI, deterministic valuation/risk/portfolio calculations, strategy screening/backtesting, reports, saved research and operational controls. The architecture must treat data lineage, ownership, authorization and reproducibility as first-class design requirements rather than UI enhancements.

Decision	Recommended design	Reason / constraint
Application topology	Modular monolith: Next.js/React web app; FastAPI API/application layer; separately runnable worker process sharing domain packages.	Matches the source stack and supports fast coordinated delivery without premature service sprawl.
Relational source of truth	PostgreSQL for identity, workspace permissions, market/reference data, document metadata, calculations, jobs and reports.	Declared schema authority; relational constraints and provenance provide reliable business records.
Vector retrieval	PostgreSQL + pgvector for chunk embeddings in the same deployment; relational chunk/document IDs remain authoritative.	Schema reference explicitly treats vector search as a logical responsibility within the PostgreSQL deployment.
External adapters	Typed adapters for market-data providers, embedding models, and LLM inference; Ollama is the initial inference runtime.	Provider replacement, deterministic tests, entitlement checks, model compatibility and graceful degradation.
Long-running work	Persist job/run state in PostgreSQL; dispatch to a worker. Redis is optional for queue/cache, never the authoritative record.	Supports retry, idempotency, execution history, observability and recovery.
Financial correctness	Python/backend calculators own numeric calculations; the LLM explains backend results and assumptions.	Prevents generated text from becoming an unaudited financial source of truth.
Authorization	Authenticate once at the API boundary; enforce resource/operation permissions inside every protected service/repository path. Filter RAG evidence before model context.	SRS requires backend enforcement and permission filtering before private content reaches the model.
Release control	Release 1: read-only market research and analysis. FS-07-04 and FS-08-04 remain Release 2 only.	No broker order-execution path or optional trading schema in Release 1.

What success means

- A user can authenticate and only access permitted account/workspace resources.
- Market data is stored with provider, timestamp, snapshot, quality and source lineage; retries do not create duplicate accepted records.
- A research document can be safely parsed, chunked, embedded, searched and removed/re-indexed without stale unauthorized vectors becoming visible.
- An AI answer can be traced to the model/configuration, authorized chunks, tool outputs and data snapshots used to produce it.
- Portfolio, DCF, risk and backtest outputs are reproducible from stored versions, inputs, assumptions, costs and run metadata.
- Reports and saved assets preserve ownership, source citations, as-of context and controlled export/sharing permissions.
- CI, unit/integration/system tests, vulnerability checks, telemetry, backup/restore and rollback provide evidence of operational readiness.

Schedule reality check

The four-sprint allocation schedules 72 story/task entries and 251 original story points (63 / 60 / 63 / 65 SP). The source documents do not establish team velocity that would make 251 SP in four calendar days achievable. Treat the sprint split as the required planning baseline; deliver only what has passed full acceptance criteria, and report unfinished items transparently.

2. Source baseline, scope and release boundaries

The source order used for technical decisions is: (1) SRS for functional/non-functional requirements and page relationships; (2) core/optional SQL schema and database reference for canonical entities, keys and relationships; (3) Jira backlog for grouped

user stories, priorities, story points and acceptance criteria; (4) four-sprint plan for current assignment of every story; (5) Sprint 1 test plan for the testing structure and initial acceptance evidence. This document proposes an implementation design; it does not claim these components have already been built or deployed.

Source	How this plan uses it	Decision boundary
49-page SRS	Nine application areas P1-P9; FR/NFR identifiers; security, RAG, financial analysis, performance and operations requirements.	SRS remains authoritative if wording differs from a Jira summary.
Complete Jira backlog	72 grouped stories/tasks in 11 epics E1-E11; titles, source refs, priority, points and acceptance checks.	Use these story entries for scope; do not resurrect the older seven-day MVP-only split.
Full four-sprint plan	Assigns all 72 story aliases uniquely to Sprints 1-4, with 23/17/17/15 entries and 63/60/63/65 SP.	Sprint allocations and source release labels must be retained.
Database Schema & ERD Reference	158 core tables plus 15 optional trading tables; domain counts, PK/FK relationships and implementation notes.	The complete schema is a target-state design; live migration success is not claimed by the source.
Sprint 1 Test Plan	54 test cases, testing rules, Sprint 1 story-to-test traceability and release boundary checks.	Results remain Not Run until observed; record evidence rather than assuming Pass.

2.1 Release 1 target

- Historical daily US-equity data as the defined baseline; other markets are supported only when configured provider coverage and permissions permit.
- Identity/access control; dashboard foundations; market ingestion, validation, snapshots and lineage; document extraction and indexing; authorized RAG; Ollama behind an adapter; AI Financial Analyst and approved deterministic tools.
- Portfolio, DCF, quantitative risk, screening/backtesting, saved research and reporting according to story priority and acceptance criteria; asynchronous jobs and observability are cross-cutting.
- No live broker order submission or broker execution in Release 1. Historical prices must not be presented as real time; each data panel must expose source and as-of/freshness context.

2.2 Release 2-only features

Hard release gate

The source Jira catalog labels FS-07-04 (paper trading/broker execution controls) and FS-08-04 (broker modes and trading safety) as Future - Release 2 only. They appear in Sprint 4 of the full allocation for catalog completeness; assignment does not authorize implementation or deployment in Release 1. Keep the optional 15-table trading DDL unapplied and do not expose order endpoints.

3. Product domains and architecture requirements

SRS area / epic	Responsibility	Primary stories / outputs
P1 / E1 - Identity, access and security	Authentication, sessions, role/permission matrix, resource ownership, audit and account protection.	FS-01-01..05; FS-09-03/05; all protected API routes.
P2 / E2 - Dashboard, market views and watchlists	Health/activity, market summary, charts, instrument comparisons, watchlists, portfolio and risk presentation, freshness.	FS-02-01..08; dashboard is a consumer of domain services, not a calculator.
P3 / E3 - AI Financial Analyst	Conversation lifecycle, streaming/cancel/retry, tool routing, source citations, safe numeric tool outputs and clear limitations.	FS-03-01..07; keep model output separate from authoritative calculation results.
P4 / E4 - Research library and RAG	Upload validation, parsing, extraction, chunking, embeddings, search, permissions, deduplication, re-indexing and lifecycle.	FS-04-01..06; document metadata and grants in PostgreSQL, vectors in pgvector.
P5 / E5 - Market ingestion and Data Explorer	Provider registration, historical prices/fundamentals/events, query safety, transformations, quality, snapshots and lineage.	FS-05-01..06; adapters normalize external records into canonical instruments/time/currency.

SRS area / epic	Responsibility	Primary stories / outputs
P6 / E6 - Strategy, screening and workflow studio	Workflow definitions, versions, approvals, strategy rules/variants, signals, alerts and scheduling.	FS-06-01..06; allowlisted operations only and immutable run-version references.
P7 / E7 - Execution monitoring and backtesting	Run history, status transitions, diagnostics, retries/cancellation, backtest metrics, comparison and exports.	FS-07-01..05; logs/run IDs are correlated, scoped and redacted.
P8 / E8 - Models/providers/settings	Ollama, embedding and retrieval configuration, provider coverage, defaults, versioning and safe configuration activation.	FS-08-01..05; validate compatibility before activation; broker modes remain gated.
P9 / E9 - Account/workspace/saved research	Profile/preferences, memberships, saved assets, explicit sharing grants, export/deletion and privacy controls.	FS-09-01..05; ownership checks are server-side.
Cross-cutting / E10 - Quant finance	DCF, portfolio snapshots/returns/attribution, VaR/CVaR, Monte Carlo/stress, reports and saved research lineage.	FS-10-01..07; all results retain assumptions and input snapshots.
Cross-cutting / E11 - Engineering and operations	Repo reproducibility, CI/CD, TDD, security scanning, architecture docs, traceability, release baselines, health and resilience.	FS-11-01..12; release evidence is part of Definition of Done.

4. Logical system architecture

The architecture separates user interaction, API policy enforcement, domain ownership, persistence, external integrations and asynchronous execution. The diagrams describe logical responsibilities, not a mandate to deploy every module as a separate microservice.

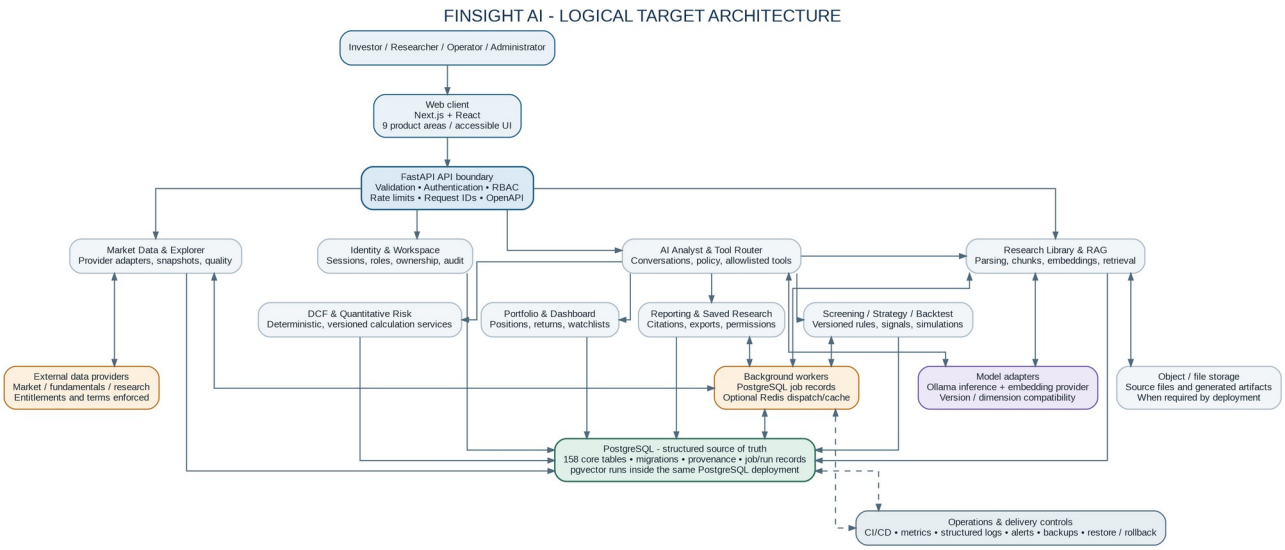


Figure 1. Logical target architecture. Domain modules can initially share one FastAPI deployment; workers may run as a separate process from the same codebase.

4.1 Layer responsibilities

Layer	Responsibilities	Design constraints
Client - Next.js/React	Nine user-facing areas: access, dashboard, AI analyst, research, Data Explorer, workflows, execution monitor, settings, account/workspace.	Presentation only; never authoritative for permissions, pricing formulas, secrets or validation.
API/application - FastAPI	HTTP boundary, request parsing, authentication, policy checks, rate limiting, orchestration, stable errors and OpenAPI.	Version public routes; validate inputs and enforce authorization on the server on every call.

Layer	Responsibilities	Design constraints
Domain services	Identity; market/reference data; document/RAG; AI/tool router; portfolio/analytics; DCF/risk; strategies/backtest/workflows; reporting; configuration.	Business logic stays in domain modules; cross-domain calls use typed service interfaces rather than table-level shortcuts.
Persistence	PostgreSQL relational data and job records; pgvector extension for embeddings; file/object storage for original uploads/artifacts where needed.	Relational metadata is source of truth; vector index and file storage must reconcile with relational lifecycle.
Integration adapters	Market-data providers, document inputs, Ollama inference and embedding models, optional notification/object-storage providers.	Timeouts, retries, entitlements, model version/dimension, and provider errors are explicit.
Workers / operations	Parsing, backfill/refresh, embeddings/indexing, scheduled analysis/backtest, reports, retries, health/metrics/logs.	Every job has durable execution metadata and correlation IDs; bounded retry and idempotency required.

4.2 Key architectural invariants

- Authorization happens before data access and before private retrieval context is assembled.
- PostgreSQL is authoritative for account ownership, instrument identifiers, snapshots, document metadata, run inputs/results and permissions.
- Financial numbers originate from deterministic backend calculations or persisted sourced values; the LLM must not invent authoritative numeric results.
- Every derived output records its as-of date/time, source/data_snapshot_id, parameter/configuration/model version and limitations.
- Provider/model failure must produce a controlled state; it must not corrupt persisted conversation history or unrelated features.
- Broker/order workflows are a separate safety domain and remain absent/disabled in the Release 1 runtime.

5. Module boundaries and repository structure

5.1 Recommended deployable shape

Use a modular monolith for the API and domain services, plus an asynchronous worker process using the same domain packages. Share contracts and domain services, not database credentials or secrets in the browser. This matches the Jira backlog instruction to start as a modular FastAPI application and avoid premature microservices.

```
repo/
  apps/
    web/                  # Next.js/React pages, client state, accessible components
    api/                  # FastAPI app, routers, auth middleware, OpenAPI
    worker/               # Background job entrypoint / consumers
  packages/
    domain/
      identity/           # Sessions, RBAC, ownership, audit
      market_data/        # Provider ports, normalization, quality, snapshots
      research/           # Upload, extraction, versions, chunking, lifecycle
      retrieval/          # Embeddings, pgvector search, access-filtered context
      ai_analyst/         # Conversation service, model orchestration, tool allowlist
      portfolio/          # Transactions, positions, snapshots, returns
      valuation/          # DCF models, assumptions, sensitivity outputs
      risk/               # VaR/CVaR, drawdown, simulation and stress methods
      strategy/           # Screening, strategy versions, variants, signals
      backtesting/        # Simulation engine, costs, metrics, integrity checks
      reporting/          # Report assembly, citations, exports, access grants
      jobs/               # Job state, retry policy, idempotency, run tracing
      configuration/      # Provider/model/index/risk configuration versions
    adapters/
      postgres/           # Repositories and transaction boundaries
      market_providers/   # Provider-specific adapters
      llm/                # Ollama + future inference adapters
      embeddings/         # Embedding adapters and dimension checks
      storage/            # Local/object storage abstraction
```

```
  notifications/          # Email/in-app/webhook adapter, if enabled
 contracts/              # Pydantic request/response DTOs and error schema
migrations/              # Versioned PostgreSQL DDL / migration scripts
tests/                  # unit, integration, contract, security, E2E, fixtures
infra/                  # containers, CI/CD, deploy config, dashboards/runbooks
docs/                   # ADRs, API guide, RTM, test evidence, runbooks
.env.example              # placeholders only; never real secrets
```

5.2 Module ownership contracts

Module	Owns	Must not
Identity / Access	User/session identity; role permissions; workspace membership; grants; audit events.	Delegate authorization to UI visibility or trust caller-provided user/workspace IDs.
Market Data	Provider registry/coverage, instrument resolution, ingestion batch, raw payload metadata, normalization, quality, snapshots, price/fundamental data.	Report unsupported coverage as complete; overwrite history silently; claim live data without entitlement.
Research / Retrieval	Document identity/version/status, extraction artifacts, chunk metadata, embeddings, index lifecycle, retrieval traces/citations.	Send results to a model before applying permission filters.
AI Analyst / Tool Router	Conversation lifecycle, prompt policy, model adapter, approved tool selection, explanation and citation assembly.	Execute arbitrary tool names or use model-generated arithmetic as a canonical metric.
Quant Finance	DCF, portfolio returns/attribution, risk and backtest deterministic math with versioned inputs.	Hide missing-data assumptions, silently switch data snapshots or mutate historical run inputs.
Workflow / Jobs	Run/job state machine, queueing, retries, cancellation, timeouts, scheduling and correlated operations data.	Use Redis/cache as the only source of execution state or retry non-idempotent side effects blindly.
Reporting / Saved Research	Report definition, source citations, snapshot references, export status, object storage pointers and sharing grants.	Expose artifacts through unscoped URLs or allow a report to outlive authorization silently.

6. Core application flows

6.1 Market-data ingestion

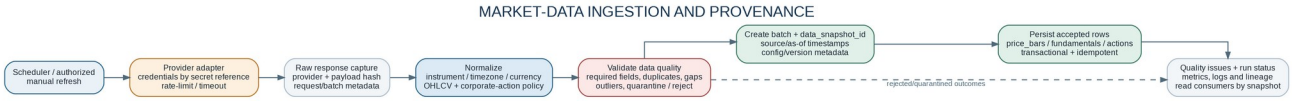


Figure 2. Ingestion path from source adapter through normalization, quality checks and snapshot-based persistence.

1. Create an ingestion batch/run with stable ID, source/provider reference, selected instrument universe and the configured as-of/interval policy.
2. Fetch via the provider adapter with bounded timeouts, rate-limit handling, request correlation and payload/hash capture where required by the design.
3. Normalize identifiers, timestamps/timezones, currency and OHLCV fields; apply explicit corporate-action/adjustment policy and retain raw-to-normalized lineage.
4. Run validation: required fields, duplicate keys, missing/gap detection, outlier checks, non-sensical price/volume ranges and provider schema drift.
5. Persist accepted rows and data_snapshot metadata transactionally where possible. Reject or quarantine bad records with a quality-issue reason and traceable batch linkage.
6. Enforce idempotency using canonical provider/instrument/time/interval identity and constraints; only accepted records are eligible for downstream analytical snapshots.
7. Expose source, data mode, as-of timestamp, freshness/quality status and coverage limitations to API consumers and the dashboard.

6.2 Document ingestion and RAG-enabled answer

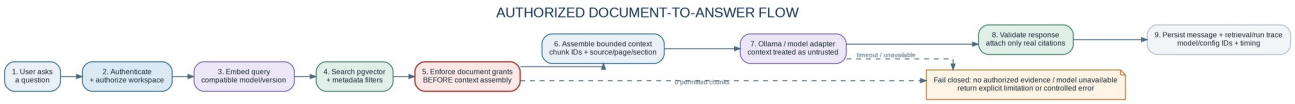


Figure 3. Retrieval permissions are applied before private evidence enters the model context; citations must resolve to real chunks/sources.

1. Validate file type, size, content and authorization; compute a content fingerprint and create a durable document/version record before queueing work.
2. Extract text and supported tables. Preserve source URL/provider, publication date, ticker/company resolution, page/section and parser/version metadata where available.
3. Chunk by section/page with deterministic chunk IDs. Generate embeddings using a configured adapter and persist model name, model version, vector dimension and index version.
4. Write embeddings to pgvector linked to stable chunk/document/version IDs; mark processing state. Re-index and delete operations must be idempotent and reconciliation-aware.
5. For a user query, authenticate, apply workspace/document grants, embed the query using the compatible model, search pgvector and filter unauthorized/deleted/superseded chunks before context assembly.
6. Use the configured LLM adapter with bounded context and explicit instructions treating retrieved text as untrusted evidence, not executable instructions.
7. Assemble the answer only with source references actually returned by retrieval. Persist conversation/message IDs, retrieval run/chunks, model/config version, status, timings and limitations.

6.3 Deterministic financial Q&A

The AI layer classifies intent and calls explicit allowlisted services. For price/return, company comparison, fundamentals, technical indicators, portfolio analysis, DCF or risk, the service returns a typed result that includes units, date range, as-of timestamp, data snapshot, formula/method and quality limitations. The LLM may summarize or explain this payload, but must not recompute/alter its numbers. Unsupported instrument resolution or missing data must trigger clarification or an explicit unavailable result.

6.4 Workflow, backtest, risk and report flow

1. Validate workflow/strategy schema and allowed tools; publish an immutable version before a run can reference it.
2. Create a durable execution_run/background_job record with stable ID, owner/workspace, actor, source/data snapshot, strategy or model configuration version, parameters and status.
3. Use a worker for long-running ingestion, indexing, risk, backtest and export tasks. Store per-step status and structured diagnostics; retry only retryable failures with bounded backoff.
4. Persist deterministic outputs: run summary, metrics/time series, costs/assumptions, warning/limitation records and all required provenance.
5. Generate a report from authorized persisted results and citations; store report access grants, input snapshots, export state and object-store URI if a file is generated.
6. Every retry/cancel/status transition is auditable; dashboard and execution monitor only display resources the requesting actor can access.

7. API and service contract design

7.1 HTTP/API conventions

Contract area	Baseline rule
Base path	Version public APIs under `/api/v1`; internal module calls may use typed Python service interfaces, not HTTP loopbacks by default.
Authentication	Use secure session or bearer mechanism consistent with deployment; set secure, HttpOnly, SameSite cookies where cookie sessions are used. HTTPS in deployed environments.
Authorization	Enforce endpoint permission plus object/workspace ownership on every request. Deny by default; never rely solely on hidden buttons.

Contract area	Baseline rule
Validation	Pydantic request/response models; explicit enum/range constraints; reject extra or unknown fields when safe to do so. Parameterized SQL and repository methods only.
Pagination	Cursor or stable limit/offset pagination for histories/search; define maximum limit and deterministic ordering.
Idempotency	Accept idempotency keys for retryable mutating/job-launch operations where duplicate request effects are possible. Combine with DB uniqueness and worker idempotency.
Errors	Consistent error object: code, safe message, request_id/correlation_id, field errors where applicable; do not expose stack traces or secrets.
Observability	Return request/run/job IDs for async work. Record actor/resource/action/status and duration; redact secrets and unnecessary private prompt text.
API docs	Generate OpenAPI from FastAPI and keep request/response examples, auth scopes, status codes and version/migration notes in docs.

7.2 Representative resource/API surface

Domain	Representative endpoints (illustrative contract)	Behavior / notes
Auth	POST `/api/v1/auth/login`; POST `/auth/logout`; GET `/auth/session`; POST `/auth/recovery`	Exact endpoints should be aligned with the current repository; generic errors; audit events; CSRF protections if cookie-based.
Users / workspaces	GET/PATCH `/users/me`; GET/POST `/workspaces`; POST `/workspaces/{id}/members`; GET `/audit-events`	Membership/role management guarded by server-side permission policy.
Health / jobs	GET `/health/live`; GET `/health/ready`; GET `/jobs/{id}`; POST `/jobs/{id}/retry`; POST `/jobs/{id}/cancel`	Separate liveness from dependency readiness; action endpoints check job ownership and retryability.
Market data	GET `/instruments`; GET `/instruments/{id}/bars`; POST `/ingestion-runs`; GET `/ingestion-runs/{id}`; GET `/data-snapshots/{id}`	Query responses include source, interval, data mode, as-of and freshness/quality information.
Documents	POST `/documents`; GET `/documents/{id}`; POST `/documents/{id}/reindex`; DELETE `/documents/{id}`; GET `/documents/search`	Uploads are validated and queued. Metadata, chunk previews and search are permission filtered.
RAG / AI	POST `/conversations`; POST `/conversations/{id}/messages`; GET `/conversations/{id}`; GET `/retrieval-runs/{id}`	Streaming is optional by inference adapter support; citations map to known document/chunk IDs.
Tools / finance	POST `/analysis/financial-questions`; POST `/valuations/dcf-runs`; POST `/risk-runs`; POST `/screen-runs`	Numeric tool outputs are deterministic and typed. Run parameters/assumptions and snapshot references persist.
Strategy / backtest	POST `/strategies`; POST `/strategies/{id}/versions`; POST `/backtest-runs`; GET `/backtest-runs/{id}`	Backtest records strategy version, data snapshot, costs, benchmark, parameters, status and outputs.
Portfolio / reports	POST `/portfolios`; POST `/portfolios/{id}/transactions`; GET `/portfolios/{id}/snapshots`; POST `/reports`; GET `/reports/{id}`; POST `/reports/{id}/exports`	Writes and exports are ownership/grant checked; report captures source citations and as-of context.
Settings	GET/PATCH `/settings/models`; POST `/settings/models/test`; GET/PATCH `/settings/providers`; POST `/settings/providers/test`	Validate before activation; secrets never returned; config version records actor/time.

These paths are a proposed API surface, not a claim that matching route handlers already exist. Lock final route names and payload schemas in API design records before implementation.

8. Data architecture and schema strategy

Canonical storage rule

PostgreSQL is the structured source of truth. The schema package defines 158 core tables and 15 optional Release 2 trading tables. pgvector is an extension inside the same PostgreSQL deployment in the target design; it is a distinct retrieval responsibility, not an implied separate physical database.

8.1 Schema domain allocation

Domain from schema reference	Declared tables	Primary responsibilities / representative entities
Identity, access and workspace	24	users, roles, role_permissions, workspaces, workspace_members, auth_sessions, password_reset_tokens, api_credentials, audit_events, access grants.
AI configuration and operations	16	AI providers/models and versioned configuration, approved tools, execution/run history, background jobs and monitoring.
Market reference, ingestion and fundamentals	31	companies, instruments, exchanges/providers/source registry, ingestion batches, raw data records, data_snapshots, price_bars, financial statements/facts, corporate actions and data quality.
Data Explorer and lineage	6	user datasets, source/schema metadata, saved queries, transforms, lineage and export records.
Research library and RAG	21	research sources/documents, document versions, chunks, document tables, chunk_embeddings, knowledge indexes, retrieval_runs and retrieval_run_chunks.
Portfolio and dashboard analytics	12	portfolios, portfolio_transactions, portfolio_snapshots, portfolio_positions, cash balances, return series, watchlists, dashboard layouts/widgets.
DCF valuation and quantitative risk	22	valuation model versions/runs/assumptions/forecasts/sensitivity; risk configs/runs, metric results, time series, covariance and stress results.
Screening, strategies, workflows and backtesting	26	saved screens/results, strategies/versions/variants, technical indicator definitions, signals, workflows, backtest_runs/trades/equity points/metrics.
Optional trading and broker integrations	15	broker providers/connections/accounts, paper-trading accounts, orders/events/fills, risk checks, approvals, positions and reconciliation; Release 2 only.

8.2 Persistence and transaction conventions

- Use canonical UUID primary keys and declared foreign keys from the schema package; preserve unique/composite constraints that encode natural identities and idempotency.
- Use timezone-aware timestamps for event, source publication, as-of, created/updated, job/run start/completion and validity times. Distinguish event time from ingestion/recorded time where the model requires it.
- Use fixed-precision numeric types for price, quantity, currency, valuation and risk outputs; never use binary floating point as the persisted source for monetary totals where decimal precision is required.
- Use transactions for relational state transitions and output writes. Since PostgreSQL and external providers/vector writes are not one atomic transaction, track the workflow state and use retry/reconciliation to recover partial failures.
- Index common access paths: ownership/workspace, instrument + interval + time, data_snapshot_id, source/provider, document/version/status, retrieval lookup keys, execution_run_id/job status and report ownership. Validate actual index definitions against query plans.
- Prefer append-only/versioned records for configuration changes and calculation runs. Do not update historical outputs in place when a source, snapshot, strategy or calculation method changes.

- Apply migrations in version control. Review SQL, test migrate-forward on a disposable database, validate foreign keys and indexes, record rollback/forward-fix strategy, and run restore tests.

8.3 Data lineage model

Every downstream market-derived or research-derived result should identify the exact snapshot and versions that were used, rather than resolving latest records at read time. For a financial result, the minimum lineage envelope is: actor/workspace; source_registry/provider; data_snapshot_id/as-of timestamp; calculation/model/config version; parameter/assumption set; run ID; outputs; limitations/warnings; and citations/links where research evidence contributed.

Artifact	Required trace fields	Failure / lifecycle consideration
Market snapshot	source_registry_id/provider, ingestion_batch_id, data_snapshot_id, as_of/interval, quality summary, parent snapshot where applicable.	Partial provider outages do not mark partial snapshots complete; rejected rows are traceable.
Document/embedding	research_document_id, document_version_id, document_chunk_id, owner/workspace, page/section, content hash, embedding model/version/dimensions, index status.	Deletion/re-indexing must reconcile relational metadata and vectors; stale/superseded chunks are not retrievable.
AI retrieval/answer	conversation_id/message_id, retrieval_run_id, authorized chunk IDs, rank/score where supported, model/config version, timings/status.	Citation must resolve to the evidence presented; no citation fabricated by the LLM.
Valuation/risk/backtest	run ID, source/data snapshot, model/strategy/config version, assumptions/seed/frequency/cost model, as-of date, status and limitations.	Re-run must be reproducible with the referenced versions; never silently use latest inputs.
Report/export	report_id, source citations, report_data_snapshots, creator/owner/workspace, access grants, artifact URI/MIME type, export status.	Authorized users only; object artifact retention/deletion consistent with report access and policy.

9. Market-data ingestion and data quality

9.1 Provider adapter contract

```
MarketDataProvider (typed interface)
  discover_coverage() -> Coverage
  fetch_instruments(query, market, as_of_policy) -> InstrumentBatch
  fetch_daily_bars(instrument_ids, date_range, adjustment_policy) -> RawBatch
  fetch_fundamentals(instrument_ids, periods) -> RawBatch  # when supported
  fetch_corporate_actions(instrument_ids, date_range) -> RawBatch  # when supported
  health_check() -> ProviderHealth

Adapter result envelope:
  provider_id, request_id, fetched_at, source_as_of, data_mode,
  entitlement/coverage note, raw_payload_hash, normalized records,
  warnings, provider cursor / continuation metadata
```

9.2 Validation rules

Rule class	Required checks	Result handling
Structure	Required fields, type and format, instrument key, bar interval, timestamps, currency codes, supported provider schema version.	Reject invalid record or whole batch depending on configured policy; preserve reason codes.
Domain sanity	OHLC relationships (high >= open/close/low; low <= open/close/high), non-negative volume where applicable, valid dates, price bounds, currency/unit consistency.	Record quality issue and quarantine clearly malformed values; do not silently coerce material errors.
Completeness	Expected trading-calendar observations, missing intervals, duplicated instrument/time keys, missing fundamental periods.	Differentiate data absence from ingestion failure; report coverage and gap counts.

Rule class	Required checks	Result handling
Consistency	Provider vs canonical ticker/instrument mapping; corporate-action adjustment strategy; snapshot/symbol identity consistency.	Do not merge ambiguous instruments automatically; request mapping/clarification or hold record.
Provenance	Provider ID, source timestamp, ingestion time, snapshot, policy/version, hash, quality status.	No sourced value is displayed without as-of/freshness context when the SRS requires it.

9.3 Consumer contract

Market-data consumers should request explicit instrument identifiers, interval/date windows and optionally a data_snapshot_id. API responses carry `data_mode`, `as_of_at`, `source`, `data_snapshot_id`, currency/unit, adjustment policy and freshness/quality state. If only daily EOD coverage is configured, the UI and model must never label it “real time.” Provider coverage, price rights, storage rights, display and redistribution terms must be reviewed before production use.

10. Document processing, RAG and AI safety

10.1 Ingestion / indexing state machine

```
RECEIVED -> VALIDATED -> QUEUED -> EXTRACTING -> CHUNKING
      -> EMBEDDING -> INDEXING -> INDEXED

Any stage may transition to FAILED with a stable error code and run ID.
Retry -> eligible FAILED state (idempotent, with prior attempt retained)
Delete -> DELETE_PENDING -> relational/vector reconciliation -> DELETED
Re-index -> create/activate compatible index version; supersede old vectors only after consistency checks
```

10.2 Retrieval algorithm

1. Authenticate the user and resolve the authorized workspace/resource scope.
2. Normalize the question and generate its embedding through the configured embedding adapter. Refuse index/query mismatches in model version or vector dimensions.
3. Run vector/hybrid search with tenant and metadata constraints where possible; independently apply the authoritative document-access predicate before constructing model context.
4. Exclude deleted, superseded, processing, or inaccessible document versions. Rerank only permitted candidates; cap top-k and context size.
5. Construct context with chunk IDs, document/version IDs, source URI, page/section/table coordinates and timestamps where available.
6. Call the LLM through the inference adapter with prompt-injection defenses, tool policy and response schema constraints. Retrieved text is untrusted data.
7. Validate citation IDs against the retrieved context; persist retrieval_run_chunks and the answer-to-source relationship. If no permitted relevant evidence exists, tell the user rather than inventing sources.

10.3 Model adapter and tool interface

Concern	Design requirement
Inference adapter	Typed interface for generate/stream/cancel where provider supports it; explicit timeout, max token limit, model identifier and normalized errors.
Embedding adapter	Separate from generation; fixed model ID/version and output dimensions. Re-indexing policy is mandatory if incompatible settings are activated.
Tool allowlist	Each tool defines input schema, required permission, resource scope, timeout, side-effect category and output schema; reject unregistered names.
Numerical finance tools	Return typed values with formula/method, unit, date period, as-of, input snapshot and warnings. LLM is not allowed to become source of numerical truth.

Concern	Design requirement
Prompt safety	Bound user and retrieved content, treat retrieval as hostile/untrusted, prevent secrets in prompts/logs, and enforce authorization in service code outside model decisions.
Conversation persistence	Persist user messages before or with generation according to defined transaction semantics; distinguish complete, streaming, cancelled and failed states; retries do not duplicate user messages.
Provider outage	Controlled error status, retry only where safe, preserve conversation history and allow unrelated services to continue.

11. Financial calculation correctness and reproducibility

11.1 Deterministic calculation services

Domain	Design responsibilities	Persisted lineage / invariants
Portfolio	Apply transaction ledger, cash balances, positions, corporate actions and price snapshots to compute holdings, market value, returns and allocation.	Portfolio snapshot, data_snapshot_id, as-of timestamp, currency, input transaction IDs, valuation status and warnings. No portfolio total without reconcilable inputs.
DCF valuation	Compute forecast cash flows, WACC, terminal value, enterprise/equity value and per-share result; expose sensitivity analysis where implemented.	valuation model version, company, input snapshot, assumptions, forecast periods, formula/version and result limitations.
Quantitative risk	VaR/CVaR, drawdown, covariance, Monte Carlo, stress scenarios, correlation and risk contributions as supported by available data/method.	risk_run_id, portfolio snapshot, data snapshot, config, lookback, return frequency, confidence level, covariance method, simulation count and random seed where relevant.
Screening/signals	Evaluate supported filter/indicator rules against explicit input snapshot and instrument universe.	Versioned rules, data snapshot, observation time, signal explanation and rule-status history; signal never means an order was submitted.
Backtesting	Simulate versioned strategy on historical snapshots with explicit fills/execution convention and transaction costs/slippage.	Strategy version, data snapshot, date range, benchmark, initial capital, cost model, assumptions, bias checks, equity curve, trades and metrics.
Reporting	Assemble persisted domain outputs, tables/charts, citations and source/input provenance.	report_data_snapshots, report citations, owner/workspace, export run state and artifact link.

11.2 Anti-look-ahead and financial integrity

- Prevent future data leakage in backtests: each signal uses only data available at the simulated timestamp; document corporate-action and survivorship handling.
- Preserve historical configuration and data snapshot references; do not rerun old reports with today's data unless explicitly creating a new version.
- Use clear conventions for market calendar, timezone, day-count, annualization, return frequency, benchmark alignment, dividend/adjustment policy and currency conversion.
- Label missing financial facts, stale data, unsupported formulas, missing portfolio inputs or insufficient observations as limitations instead of silently substituting data.
- Validate calculators with deterministic fixtures and hand-calculated reference cases; use numeric tolerances appropriate to each financial measure.
- Keep calculation outputs separate from model-generated narrative; the response generator cites backend output and does not alter numbers.

12. Security, privacy and threat controls

Threat / failure mode	Preventive control	Verification evidence
Unauthorized API / broken access control	Default-deny RBAC, policy checks at API and domain service, resource owner/workspace checks, least privilege.	Anonymous/unauthorized/other-tenant API tests; permission matrix; audit events.
Credential or provider-secret leak	Environment/secrets manager; DB stores refs/hashes rather than plaintext secrets; secret scanning; no secrets in UI/logs/examples.	Secret scan and log inspection; negative tests assert secrets never returned.
Session compromise / brute force	Secure cookie/token policy, expiry/invalidation, rate limit/temporary lockout, optional MFA per deployment policy.	Login negative tests, logout/replay and expiry tests, throttling tests.
Prompt injection / malicious documents	Retrieved text marked untrusted; allowed tools only; context limits; parser isolation; server-side permission check before LLM.	Prompt-injection suite; tool allowlist tests; unauthorized chunk never reaches model context.
SQL injection / unsafe Data Explorer	Parameterized SQL or constrained query builder, query allowlist/read-only role, statement timeout and row caps.	Injection tests, query planner inspection and read-only DB role verification.
Cross-workspace information leak	Workspace/owner scoping in repositories and retrieval; object storage paths not exposed unguarded.	Negative tests across documents, conversations, jobs, reports and exports.
Stale/orphaned vectors or artifacts	Explicit lifecycle state machine, idempotent re-index/delete, reconciliation jobs, storage retention policy.	Re-index/delete partial-failure integration tests; orphan detection reports.
Data entitlement/license breach	Provider coverage and use restrictions in configuration; record source; gate export/display by license/entitlement.	Provider terms review and source capability tests; source licensing checklist.
Unintended live trade	No Release 1 order endpoints/secrets/table extension; explicit release flags and separate approval/step-up checks in future Release 2.	Route/schema/deployment scan confirms R2 DDL and order path are absent or disabled.

12.1 Security implementation baseline

- Use TLS/HTTPS in deployed environments; secure cookie attributes and CSRF control if browser cookies authenticate mutating requests.
- Hash passwords with a maintained password-hashing implementation; do not log credentials, tokens, private document text or full sensitive prompts.
- Enforce resource authorization on reads as well as writes, exports, downloads, job detail pages, RAG sources and report citations.
- Use validated uploads, strict type/size limits, isolated parsing for risky formats and output encoding in the UI.
- Run dependency vulnerability checks, SAST/static analysis, secret scanning and license review in CI; critical auth, data-corruption, secret exposure or exploitable injection defects block release.
- Generate and review an SBOM where tooling permits; document retention, backup, restore and incident-response ownership.

13. Background jobs, reliability and observability

13.1 Job execution contract

Field / rule	Implementation
Stable identity	Persist job_id/execution_run_id; add request correlation ID and parent run where nested steps exist.
State machine	queued, running, completed, failed, cancelled; document allowed transitions and terminal-state behavior.
Attempt history	Persist job_attempts, retry count, timing, sanitized error code/message and step trace; do not replace previous failure evidence.
Idempotency	Persist idempotency key/operation identity; use upserts/unique constraints or dedupe records for source/instrument/time or document-version/chunk IDs.

Field / rule	Implementation
Retry policy	Retry transient connection/timeouts with exponential backoff and jitter; cap attempts and elapsed time; no retry for invalid data, authorization failures or non-retryable business errors.
Cancellation	Cooperative cancellation where provider/worker supports it; mark cancelled only after safe state is reached and persist partial output semantics.
Queue/cache	PostgreSQL is durable state. Redis may dispatch jobs or cache query data; queue contents/cache are rebuildable and non-authoritative.
Concurrency	Limit per provider/model, CPU-intensive workers and per-user job load; use bounded queues and timeouts to prevent resource exhaustion.
Notifications	If enabled, emit completion/failure notifications after persistence; notification failure must not undo the job result.

13.2 Observability model

Signal	Required shape / dimensions
Structured logs	JSON fields: timestamp, level, service/module, environment, request_id, run_id/job_id, actor/workspace ID where policy permits, event code, duration, status; redact credentials and private payload.
Metrics	Request latency/error rate by route; database connection/pool status; provider failures/rate-limit; job queue depth/duration/retry; document indexing state; retrieval latency; model call latency/tokens if available; calculation duration.
Health endpoints	Liveness answers whether the process runs; readiness checks critical dependencies needed for this instance. Optional dependency outage should not make unrelated functions appear healthy if they rely on it.
Alert conditions	Repeated provider/model outage; elevated 5xx; failed migrations; growing queue; failed indexing; repeated permission denial anomalies; backup failures; stale data beyond configured policy.
Run diagnostics	Authorized run-detail UI links status transitions and steps to correlated logs; secrets and private text are excluded from normal logs.
Recovery	Document bounded retry, reconciliation, backup/restore, rollback and provider failover/fixture behavior. Define RTO/RPO with the deployment owner; the source does not specify numerical values.

14. Testing strategy and release gates

The Sprint 1 Test Plan provides a 54-case baseline for repository/CI, authentication, configuration, credentials/privacy, operations and boundary testing. Extend the same test discipline to all remaining story acceptance criteria rather than treating the Sprint 1 cases as a complete test suite for the whole SRS.

Test level	Scope	Required examples / evidence
Static / code review	All changes and critical source boundaries.	Lint/type checks, schema migration lint/review, dependency and license scan, secret scan, SAST if available, API schema review and PR evidence.
Unit	Pure domain logic with deterministic fixtures.	RBAC decisions; validators; normalization; OHLC sanity; snapshot/idempotency keys; chunking; model compatibility; DCF formulas; risk math; backtest costs; output/citation validation.
API / contract	Request/response, schema, permissions, errors and pagination.	OpenAPI contract tests; malformed inputs; status codes; auth guards; boundary row/size limits; stable error payloads; rate limits.

Test level	Scope	Required examples / evidence
Integration	PostgreSQL + migrations, pgvector, adapters, jobs, Ollama interface and object storage.	Migration on blank and upgrade DB; ingestion transaction/retry; vector insert/search/delete reconciliation; model timeout; connection refusal; audit/job persistence.
System / E2E	Complete user journeys across frontend/API/data/model.	Sign-in and protected route; historical data ingest/query; upload->index->authorized search->cited answer; create calculation/run->report export.
Security / abuse	Trust boundaries and negative paths.	Cross-tenant reads/writes; prompt injection; unauthorized RAG chunk; SQL injection; malicious/oversized files; secret leakage; open trading route checks.
Performance	Documented reference environment; representative synthetic dataset.	Auth p95; ordinary API p95; dashboard p95; semantic retrieval p95; first token/visible AI output; backtest/batch throughput.
Reliability / resilience	Dependency loss, retry and data consistency.	Ollama down; provider timeout/429; DB connection interruption; queue restart; partial vector failure; retry idempotency; backup/restore drill.
Regression / usability	Every accepted fix and cross-page behavior.	Fixed defects linked to tests; responsive behavior; keyboard/focus, labels, understandable errors and freshness disclosure.

14.1 Automated quality gates

- Every pull request runs formatting/lint, static checks, unit tests, API tests, secret/dependency scans and frontend/backend builds.
- Integration tests run against disposable PostgreSQL with the pgvector extension for features depending on vector search; model calls can be mocked in CI and separately checked on local/staging Ollama.
- Critical invariants have negative tests: no unauthorized access, no secret output, no unauthorized chunk in LLM context, no duplicated ingestion on retry, no unsupported citation and no Release 1 order execution.
- For numeric domains, include deterministic reference cases and property/invariant checks; compare outputs within domain-specific tolerances, not arbitrary string equality.
- Coverage is reported as evidence rather than a vanity target; thresholds should be agreed per package, with priority to security, financial math, lineage and idempotency branches.
- All test records start Not Run until observed. Pass requires expected and actual match; Fail creates a defect; Blocked records the unavailable dependency and owner/next action.

14.2 Source-stated performance objectives

SRS target	Implementation interpretation	Measurement guardrails
Authentication: successful sign-in p95 < 2 seconds	Track from API receipt through session creation excluding external identity delays as stated by SRS.	Record environment, dataset and auth provider; do not count browser load time as backend latency.
Dashboard primary content p95 < 2 seconds	Measure core dashboard shell plus required primary data under agreed reference test conditions.	Test data/provider conditions and UI viewport must be stated.
Ordinary backend operations p95 < 500 ms	Applies to operations not invoking an LLM, waiting for a long job or slow external source.	Separate DB/compute from external provider/model latency.
Semantic retrieval p95 <= 1.5 seconds	Measured on agreed reference dataset/hardware, excluding embedding cold starts and downstream LLM generation.	Benchmark top-k, index/model version and warm/cold behavior separately.
Warmed local model: first visible output p95 < 5 seconds	Only on agreed hardware/model after warm-up, excluding cases explicitly identified as unusually large/cold starts.	Log model, quantization/runtime, hardware and prompt size; never promise universal LLM latency.

15. Four-sprint implementation plan based on all 72 Jira entries

Planning basis

This plan follows the current Full Project Four-Sprint Plan and the complete story catalog, not the older seven-day schedule. The original allocation is: Sprint 1 = 23 entries / 63 SP; Sprint 2 = 17 / 60 SP; Sprint 3 = 17 / 63 SP; Sprint 4 = 15 / 65 SP. All 72 entries are represented. Story aliases are supplied tracking IDs; map each one to its generated Jira issue key.

Scope-label control: the story tables summarize engineering focus; they do not supersede the original backlog priority, release/scope tag, SRS reference or full acceptance criteria. Preserve those fields in Jira. Schedule placement is not a release approval, and stories labelled later/future remain gated by their source scope.

Sprint 1 - Platform foundation, identity and delivery controls

Date: 18 October 2026 | Catalog entries: 23 | Source story points: 63 SP

Sprint goal: Create a reproducible development baseline, protected API surface, session/RBAC policy, safe configuration handling and continuous quality/security controls.

Technical workstream: Bootstrap web/API/worker packages, migration runner, contracts/error model, authentication/authorization service, configuration service and CI gates; agree ADRs before parallel coding.

Dependencies / gating: All further sprints depend on repository setup, PostgreSQL migration baseline, typed API/error contracts, identity/ownership rules and a functioning test pipeline.

Jira alias	User story / task	SP	Primary ownership	Technical implementation / acceptance focus
FS-11-01	Create reproducible repository and environment setup	2	Repo/DevEx	Clean clone, pinned dependencies, .env.example placeholders and documented startup.
FS-11-02	Run CI quality gates on every pull request	3	CI/CD	Lint/static/unit/security/build gates; demonstrate failed check blocks merge.
FS-11-03	Build and deploy a versioned release artifact	3	CI/CD	Versioned artifact, environment config and documented deploy/rollback path.
FS-11-04	Apply TDD and maintain unit/integration/E2E suites	5	QA platform	Test scaffold and critical validator/auth/idempotency/error suites.
FS-11-05	Measure line/branch coverage and test quality	2	QA metrics	Coverage report plus follow-up issues for critical gaps.
FS-11-06	Run security validation and dependency checks	3	DevSecOps	Secret/dependency/security checks; critical findings block release.
FS-11-07	Document architecture, APIs, ADRs and compatibility rules	2	Architecture docs	API/error contracts, ADRs, schema/version migration rules.
FS-11-08	Maintain SRS addendum and requirement traceability matrix	3	Requirements	Every requirement/story/test has a link; deferred items keep release labels.
FS-11-09	Run planning, daily review, demo and retrospective	2	Delivery	Daily goal, acceptance evidence, blockers and one retro action.
FS-11-10	Manage baselines, bugs, versions, SBOM and release notes	2	Release engineering	Tag/changelog/defect traceability; SBOM or explicit tooling limitation.
FS-11-11	Document usability, AI use, licenses and data rights	2	Governance	Accessibility, open-source and provider rights reviewed and recorded.
FS-11-12	Add health checks, structured logs, retry and recovery runbook	3	SRE / ops	Health endpoints, correlated/redacted logs, bounded retries and outage drill.

Jira alias	User story / task	SP	Primary ownership	Technical implementation / acceptance focus
FS-01-01	Authenticate and manage sessions	3	Identity	Valid login/logout/recovery/onboarding policy and secure session invalidation.
FS-01-02	Enforce roles and protected routes	3	Identity	Anonymous requests rejected; role/permission matrix enforced in backend.
FS-01-03	Protect accounts and audit authentication	3	Identity/security	Audit events, brute-force protection; MFA/session management per policy.
FS-01-04	Separate read-only and trading permissions	2	Identity/security	No R1 order route; separate privilege model and connection ownership checks.
FS-01-05	Meet identity security and accessibility requirements	3	Identity/security	Password hash, session/transport safeguards and accessible controls.
FS-08-01	Configure Ollama, model selection and inference parameters	3	AI configuration	Model connection/discovery/config validation and controlled unavailable state.
FS-08-05	Secure and validate configuration changes	3	Configuration	Secret references, audit, validate-before-activate and recovery to prior config.
FS-09-03	Manage API credentials, privacy, export, deletion and audit	3	Privacy/security	Credential lifecycle metadata; no secret redisplay; destructive confirmations/audit.
FS-09-05	Enforce privacy and safe account administration	3	Privacy/security	Cross-user reads/writes denied; admin privileges and retention rules audited.
FS-07-05	Secure execution logs and retain reliable run history	3	Observability	Run IDs link logs/status; scope details; retain records across UI failures.
FS-02-08	Keep dashboard responsive, isolated and freshness-aware	2	Frontend / quality	Responsive shell and authorized data scoping; freshness/error states defined.

Sprint exit gate

- Clean checkout can be built and started with documented versions and test-only environment variables.
- Anonymous and unauthorized requests are rejected in the backend; password/session data is handled securely and cross-user access fails.
- CI enforces lint/static/unit/build/security gates and retains reports; migration and rollback guidance is documented.
- Configuration secrets are never committed or returned; Ollama failure is a controlled state, not an application crash.
- Health endpoints, structured run/request IDs, redacted logs, baseline traceability and release/ethics/license records exist.

Execution rule: stories with broad acceptance criteria are not marked Done after only a partial slice. Use Jira subtasks to record partial work, attach tests/PR evidence and preserve source release labels.

Sprint 2 - Market data, research indexing and dashboard foundation

Date: 19 October 2026 | Catalog entries: 17 | Source story points: 60 SP

Sprint goal: Create the trusted data substrate: configured adapters, normalized historical daily data, quality/snapshot lineage, document extraction/chunking/vector indexing and initial dashboard coverage.

Technical workstream: Implement provider adapters and ingestion jobs; schema migrations/repos for instruments, batches, snapshots, price bars and quality issues; file parsing/chunk/embedding/indexing workers; health/market/chart APIs and first client views.

Dependencies / gating: Sprint 1 auth, configuration, repository, migration, run IDs and tests must be active. Provider credentials/entitlements or deterministic provider fixtures must be documented.

Jira alias	User story / task	SP	Primary ownership	Technical implementation / acceptance focus
FS-05-01	Register and configure data sources securely	3	Market data	Connection tests, secret-safe diagnostics and ownership/allowlist enforcement.
FS-05-02	Query structured datasets safely	3	Data Explorer	Read-only approved query path, timeouts/row limits/pagination; full expansion per scope label.
FS-05-03	Ingest market prices, instruments, fundamentals and events	5	Market data	Provider normalization, daily historical subset, source/snapshot persistence.
FS-05-04	Import, preview, clean, transform and combine datasets	5	Data Explorer	Traceable imports/transforms/joins and controlled saved datasets; preserve its source release label.
FS-05-05	Backfill and refresh data with quality and lineage	5	Market data	Idempotent backfill/refresh, data-quality outcomes, source/as-of/snapshot lineage.
FS-05-06	Enforce least privilege, query safety and source isolation	3	Data security	Minimum provider privilege, query bounds, safe secrets and audit.
FS-04-01	Upload, validate and extract documents	3	Research ingestion	Type/size/content validation and extracted text tied to document version.
FS-04-02	Chunk, embed, index and track processing	5	RAG indexing	Stable chunk metadata, embedding model/dimensions, idempotent indexing status.
FS-04-03	Search, preview, filter and manage research documents	3	Research UI/API	Permission-filtered search and previews; source links resolve to real records.
FS-04-04	Handle duplicates and recover processing failures	3	RAG jobs	Fingerprint duplicate detection, safe retry and partial-failure recovery.
FS-04-06	Protect file processing, index consistency and document lifecycle	3	RAG security	Safe uploads, isolated long jobs, reconciliation and unauthorized discovery prevention.
FS-08-02	Configure embeddings, chunking and retrieval	3	RAG configuration	Model/dimension/index compatibility, filtering and test retrieval.
FS-08-03	Manage market providers, data coverage and analytics defaults	3	Provider configuration	Coverage/entitlements visible, validated settings and versioned assumptions.
FS-02-01	Show dashboard health and activity	3	Dashboard/ops	Authorized health/activity metrics, job status and clear errors/empty states.
FS-02-02	Show market overview, sessions, movers and heatmaps	3	Market dashboard	Only configured supported data; provider/source/as-of/freshness clear.
FS-02-03	Explore interactive historical price charts	5	Frontend/market data	Supported intervals, zoom/pan and timestamps that match available data.
FS-09-01	Manage profile, security, interface and notification preferences	2	Account/workspace	Preferences persist and private fields remain isolated.

Sprint exit gate

- Market records can be ingested/re-ingested without uncontrolled duplicates; accepted/rejected quality outcomes have source/as-of/snapshot linkage.

- Document upload validates content, extracts supported formats, stores version/chunk metadata and indexes compatible embeddings in pgvector.
- Failed/retried indexing and provider calls are diagnosable, idempotent and do not leak private metadata.
- Dashboard shows service/ingestion status and clearly labels historical data mode/source/as-of/freshness; chart availability matches source coverage.
- Integration tests cover PostgreSQL migration, ingestion persistence, indexing/search and failure/retry behavior.

Execution rule: stories with broad acceptance criteria are not marked Done after only a partial slice. Use Jira subtasks to record partial work, attach tests/PR evidence and preserve source release labels.

Sprint 3 - AI analyst, authorized retrieval, strategy and workspace

Date: 20 October 2026 | Catalog entries: 17 | Source story points: 63 SP

Sprint goal: Connect the document/data foundation to source-grounded conversation and deterministic financial tools; implement workflow and strategy objects with bounded/authorized execution and saved workspace behavior.

Technical workstream: Conversation/message services, streaming adapter, permission-filtered retrieval service, citation assembly, approved tool registry, screening/strategy/workflow versioning, signal storage, watchlist/portfolio view integrations and workspace sharing.

Dependencies / gating: Sprint 2 must provide stable instruments/snapshots, indexed chunks, access grants, compatible embedding model and working model adapter. Retrieval permissions precede context assembly.

Jira alias	User story / task	SP	Primary ownership	Technical implementation / acceptance focus
FS-03-01	Create and manage AI conversations	3	AI analyst	Conversation ownership, list/search/rename/delete and configured model choice.
FS-03-02	Stream, stop and regenerate answers	3	AI analyst	Ordered streamed output, cancellation where supported, no duplicate messages on retry.
FS-03-03	Answer with authorized retrieval and precise citations	5	RAG / AI security	Only authorized chunks enter context; citations map to source/page/section metadata.
FS-03-04	Answer financial questions using deterministic tools	5	AI/finance services	Numeric results from approved backend calculators with method/time/input context.
FS-03-05	Create natural-language screening requests	3	AI/screening	Parse to displayed/validated supported conditions and reproducible snapshot results.
FS-03-06	Search, export, save and invoke approved workflows	3	AI / saved research	Ownership isolation, provenance and allowlisted tool calls; analysis never trades.
FS-03-07	Make AI behavior safe, traceable and understandable	3	AI safety	Model identity/limitations, privacy, injection defenses and controlled failure handling.
FS-06-01	Create, configure, validate, test and version workflows	5	Workflow studio	Invalid config cannot execute; immutable version and source/tool validation.
FS-06-02	Run, schedule and monitor workflows with approvals	5	Workflow / jobs	Run IDs/state transitions, bounded retry/timeouts and approval gates.
FS-06-03	Build and save stock screens	3	Screening	Validated filters, snapshot-bound result, save/clone/edit and rerun.
FS-06-04	Define versioned strategies, variants and readiness checks	5	Strategy engine	Versioned rule sets/parameters, supported conditions and readiness report.
FS-06-05	Generate explainable signals and alerts safely	3	Signals / alerting	Signal stores triggering rule, input, time/provenance; does not submit order.

Jira alias	User story / task	SP	Primary ownership	Technical implementation / acceptance focus
FS-06-06	Make workflow execution secure, bounded and reproducible	3	Workflow security	Allowlisted tools, per-run isolation, config/version inputs and idempotent retry.
FS-02-05	Manage watchlists	3	Workspace / dashboard	Owner-scoped CRUD for list/instruments with freshness metadata.
FS-02-06	View portfolio performance, allocation and risk	5	Portfolio analytics	Snapshot-aligned metrics with assumptions and missing-input states.
FS-02-07	View news, events, alerts and custom dashboard layout	3	Dashboard / saved UI	Source/time attribution, consistent filters and saved layout permissions.
FS-09-02	Manage workspaces, members, roles and sharing	3	Workspace/security	Only authorized owners administer membership; sharing requires explicit grants.

Sprint exit gate

- AI conversations persist; stream/stop/retry states are truthful and do not duplicate messages or corrupt history.
- Only authorized chunks enter model context; each displayed citation resolves to a stored source/chunk; no-evidence/model-down behavior is explicit.
- Financial numeric answers call deterministic tools and include formula/method, units, as-of date, snapshot and assumptions.
- Workflow and strategy versions, validation, run records, signals/alerts and approval gates persist; arbitrary tool execution is impossible.
- Watchlists, portfolio/risk views and workspace shares enforce owner/workspace permissions and handle missing data cleanly.

Execution rule: stories with broad acceptance criteria are not marked Done after only a partial slice. Use Jira subtasks to record partial work, attach tests/PR evidence and preserve source release labels.

Sprint 4 - Portfolio, valuation, risk, backtesting and reporting

Date: 21 October 2026 | Catalog entries: 15 | Source story points: 65 SP

Sprint goal: Complete the remaining quantitative and persistence paths: portfolio transactions/snapshots/returns, DCF, risk scenarios, backtest metrics, execution history, source-backed reports and saved research; document future trading boundary.

Technical workstream: Implement deterministic Python calculation services, run metadata and result tables, cost-aware backtesting, report assembly/export workers, saved item access grants, financial metadata/versioning and R2 trading controls only in the approved future scope.

Dependencies / gating: Sprints 1-3 provide the authenticated API, versioned configuration, market snapshots, deterministic tool pattern, strategy definitions and worker/run observability. Backtest/valuation/risk cannot be accepted without immutable input references.

Jira alias	User story / task	SP	Primary ownership	Technical implementation / acceptance focus
FS-07-01	Inspect execution history and troubleshoot jobs	3	Jobs / operations	Stable run IDs, step/status history, retry/cancel policy and secure diagnostics.
FS-07-02	Configure backtests and calculate performance metrics	5	Backtesting	Strategy version + data snapshot + costs/assumptions persisted; metrics reproducible.
FS-07-03	Analyze, compare, validate and export backtests	5	Backtesting / reporting	Equity curves/trade metrics, comparison, out-of-sample and bias limitations.
FS-07-04	Implement paper trading and broker execution controls	8	Broker/trading	Release 2 only. Separate scope; no R1 trading DDL/orders; future safeguards are documented.
FS-10-01	Calculate DCF valuation with versioned assumptions	5	Valuation	Company, data snapshot, model version, assumptions, forecasts and sensitivities persist.

Jira alias	User story / task	SP	Primary ownership	Technical implementation / acceptance focus
FS-10-02	Calculate portfolio VaR/CVaR and drawdown	5	Risk analytics	Risk config, lookback, frequency, confidence and limitations tied to input snapshot.
FS-10-03	Run Monte Carlo and stress scenarios	5	Risk analytics	Scenario parameters/config are versioned; incomplete data is explicitly flagged.
FS-10-04	Manage portfolios, transactions and snapshots	5	Portfolio domain	Transactions reconcile into positions/snapshots and retain timestamps/input references.
FS-10-05	Calculate returns, attribution and allocation	5	Portfolio analytics	Method/period explicit; results reconcile and can be regenerated from saved inputs.
FS-10-06	Generate source-backed PDF/CSV reports	3	Reporting	Report includes citations, as-of and snapshot; export status tracked and access enforced.
FS-10-07	Share, reopen and version saved research	3	Reporting / saved research	Explicit access grants and persistent source/calculation lineage.
FS-09-04	Save dashboards, watchlists, screens, strategies, reports and preferences	3	Saved research	Saved items preserve owner/workspace/time/source references and access grants.
FS-04-05	Enrich financial metadata and track document versions	5	Research lifecycle	Source/publication/ company/ticker/version/ collection lineage and grants persist.
FS-02-04	Compare instruments and fundamentals	3	Market analytics	Comparable time windows; fundamentals show source/period; missing metrics are marked.
FS-08-04	Keep broker modes and trading safety controls gated	2	Broker configuration	Release 2 only; R1 exposes no live execution; future mode changes require authorization and confirmation.

Sprint exit gate

- DCF/risk/backtest outputs are repeatable from stored versions, parameters, snapshots and assumptions; numeric reference tests pass.
- Portfolio transactions reconcile to holdings/cash/returns; input time, source and snapshot context remain attached.
- Backtesting stores strategy version, selected period/universe, cost model, benchmark, trades, equity series and bias/out-of-sample limitations where supported.
- Reports capture source citations, input snapshots, generation status and access grants; export and saved-research lifecycle are tested.
- Execution history is access-controlled and correlated with sanitized logs; regression/security tests are green or explicitly triaged.
- FS-07-04 and FS-08-04 stay marked Release 2 only. No R1 order path or optional trading DDL is enabled.

Execution rule: stories with broad acceptance criteria are not marked Done after only a partial slice. Use Jira subtasks to record partial work, attach tests/PR evidence and preserve source release labels.

16. Cross-sprint dependency graph and parallel workstreams

Dependency chain	Required contract / release gate
Identity -> every protected route	Auth/session model, policy evaluator, ownership-scoped repositories, stable 401/403 responses and audit fields.
Provider adapter -> market consumers	Canonical instrument IDs, date/timezone/currency conventions, provider/source metadata, data_snapshot_id and explicit quality/freshness states.

Dependency chain	Required contract / release gate
Document parser -> retrieval index	Document/version/chunk IDs and source coordinates, content fingerprint, processing state, compatible embedding model/version/dimension and reconcile workflow.
Authorization -> AI/RAG -> cited answer	Permission filtering before context assembly; real source/chunk citations; no-evidence path; model/config/retrieval trace.
Market snapshot + strategy version -> backtest	Frozen input snapshot, versioned strategy/rules, explicit cost model/benchmark/time range and no future data leakage.
Portfolio inputs -> DCF/risk/reporting	Snapshot-aligned positions/financial data, versioned formulas/configs, assumptions, source/as-of metadata and limitations.
Jobs -> dashboard/observability	Stable run IDs, allowed state transitions, step events, bounded retry/cancel, owner-scoped diagnostic view and redacted logs.
Release 2 safety -> no accidental R1 execution	Trading story labels, deployment flags, schema packaging and route/security checks; separate approval and security review before future enablement.

16.1 Suggested engineering workstreams

Workstream	Sprint 1-2 lead responsibilities	Sprint 3-4 integration responsibilities
Frontend	App shell, auth states, common form/table/chart patterns, API client/error/loading/empty states.	Conversation UI/streaming, research preview/citations, dashboard analytics, saved reports and accessible failure cases.
Backend / domain	Modular package boundaries, API contracts, identity/policy middleware, migration/repository patterns, configuration.	Tool router, portfolio/risk/valuation/backtest service contracts, report/workflow endpoints and permission enforcement.
Data / market	Source registry/provider adapter, normalization, snapshot model, quality/lineage, ingestion workers.	Historical query contracts, comparison/snapshot consumers, data gap/freshness presentation and calculation input checks.
AI / research	Parser and document lifecycle, embedding adapter, pgvector storage/search, Ollama adapter/health.	Permission-filtered context, streaming/citations, deterministic tools, injection/negative tests and response traceability.
QA / DevSecOps	CI test harness, DB integration environment, secret/dependency scans, baseline Sprint 1 tests.	Full vertical E2E, cross-domain regression, performance runs, deployment/rollback evidence and release readiness report.

17. CI/CD, deployment and operational runbooks

17.1 Environment topology

Environment	Purpose	Configuration and controls
Local development	Fast iteration with fixtures and disposable data.	`.env` ignored; `.env.example` placeholders only; local Postgres/pgvector; optional local Ollama; no production credentials.
CI	Deterministic regression and supply-chain quality gates.	Ephemeral PostgreSQL+pgvector; deterministic market/doc fixtures; model mocked unless dedicated inference runner is available; no network dependency to live providers.
Integration / staging	Test actual DB migrations, configured provider adapters, Ollama, object storage and E2E flows.	Synthetic/sanitized data; secret manager; TLS; restricted outbound access; deployment configuration mirrors production shape.

Environment	Purpose	Configuration and controls
Production / approved release	Authorized read-only financial research and analytics services.	Secrets manager, backups, alerts, access review, provider licensing approvals, migrations with rollback/forward fix and explicit feature flags.
Release 2 isolated validation	Only after separate approval for paper/live broker work.	Separate schema/migrations, permissions, broker sandbox credentials, order confirmation, pre-trade risk, reconciliation and kill switch.

17.2 Pull request and release pipeline

1. Checkout / dependency lock validation / secret and license scan.
2. Format, lint, static/type checks and unit tests; produce JUnit and coverage artifacts.
3. Validate migrations; run integration tests against disposable PostgreSQL + pgvector; verify clean install and upgrade from previous supported schema.
4. Run API contract/security tests, frontend build, dependency audit/SAST if available and SBOM generation where practical.
5. Build immutable image/artifact tagged by commit and version; store provenance and dependency inventory.
6. Deploy to staging using environment configuration and secret references; run smoke tests and minimal E2E tests with stable run IDs.
7. Promote only after acceptance, security and operational sign-off; perform backward-compatible migrations or documented maintenance-window migrations.
8. After deploy, run readiness checks and monitor 5xx/latency/job failures/data freshness. Roll back application image or apply a forward-fix according to migration compatibility.

17.3 Deployment / schema operational rules

- Use a least-privileged runtime DB role distinct from migration/admin roles. Do not grant arbitrary DDL or unrestricted SQL to Data Explorer paths.
- Keep schema changes version-controlled. Apply the core schema in staged migrations. The optional trading schema stays separate and is not applied to Release 1.
- Run backup schedules for PostgreSQL and the metadata necessary to rebuild pgvector indexes; verify restoration on a disposable environment. The SRS requires the deployment owner to set RTO/RPO values.
- Persist original uploads and generated artifacts in file/object storage as configured; DB stores authoritative metadata and pointers. Apply retention and delete/reconciliation rules.
- Use readiness and dependency health checks per service. Graceful shutdown stops new work and safely finishes/cancels in-flight jobs as policy permits.
- Maintain a release checklist: version, migration result, build ID, smoke report, test report, vulnerability exceptions, data/provider entitlement notes, backup status, known limitations, rollback command and owner.

18. Non-functional target matrix

Quality attribute	Design commitment	Validation / owner
Security & privacy	Backend RBAC, ownership/workspace isolation, TLS deployed, secret management, input validation, SQL injection resistance, redacted logs and audit events.	Security tests and DevSecOps gates; security owner.
Performance	SRS targets are measured on a stated reference machine, dataset, model and workload; no blanket promise independent of hardware/providers.	Performance benchmark report; QA/SRE.
Reliability	Controlled failures, bounded retries/timeouts, idempotent operations, vector/relational reconciliation, graceful shutdown.	Failure injection/outage drills; platform owner.
Data integrity	Canonical IDs, FK/unique constraints, transactional state transitions, input snapshots, immutable run/config versions.	Migration + integration + reconciliation tests; data/domain owners.

Quality attribute	Design commitment	Validation / owner
Maintainability	Modular FastAPI package boundaries, adapter interfaces, explicit error contracts, versioned migrations, pinned dependencies.	Architecture review and repository standards; tech lead.
Accessibility/usability	WCAG 2.2 AA target per SRS, keyboard/focus labels, understandable status/error/empty/freshness states, responsive supported screens.	A11y smoke checks and usability checklist; frontend/QA.
AI quality and traceability	Model identity, citations and retrieval/run trace; no unsupported guarantee; permissions before model; config/index compatibility.	RAG negative tests and evaluation fixtures; AI/research owner.
Operations	Health checks, structured logs, metrics, alerts, backup/recovery, deployment reproducibility and runbooks.	Staging acceptance / ops review; SRE/DevOps.

19. Architecture decisions, risks and open questions

19.1 Architecture decision records (ADRs)

ADR	Decision	Trade-off / revisit condition
ADR-001: Modular monolith	Keep domain services in explicit FastAPI modules; run workers as separate processes from the same domain code.	Simpler transaction/debugging and coordinated change; revisit when independent scaling, fault isolation or team ownership requires a service boundary.
ADR-002: PostgreSQL + pgvector	Use PostgreSQL as structured source of truth and pgvector for embeddings in the same deployment.	Simplifies access/provenance; benchmark indexing/recall and capacity before separating vector infrastructure.
ADR-003: Ports/adapters	Market, inference, embeddings, storage and notification systems accessed through typed adapters.	More upfront contracts, less vendor lock-in and deterministic tests.
ADR-004: Persisted job state	PostgreSQL is authoritative; Redis queue/cache is optional acceleration.	Durable audit/recovery at modest complexity; define job claiming/locking and idempotency carefully.
ADR-005: LLM is not financial source of truth	Numeric outputs from deterministic domain tools with snapshot/config provenance; LLM explains.	Some free-form questions need clarification or tool-unavailable response; protects financial correctness.
ADR-006: Release 1 read-only	No live order endpoints/optional trading schema in Release 1.	Future trading requires independent product, regulatory, security, authorization and operational approval.

19.2 Primary project risks

Risk	Impact	Mitigation / owner
251 SP scheduled across four calendar days	Most likely delivery risk; broad story acceptance criteria may not fit.	Use parallel domain owners and integration gates; report completed acceptance evidence, carry unfinished stories honestly; Product Owner decides scope/date trade.
Provider data rights/credentials unavailable	Market-data features cannot be validated against live source.	Use deterministic fixtures behind the same adapter interface and label demo/fixture data; secure provider approval before production.
Ollama / embedding model / hardware mismatch	Slow inference, missing model or incompatible index.	Record model/runtime/hardware, health check adapters, model/dimension compatibility and mock inference in CI.
RAG ACL applied too late	Cross-user private document disclosure through LLM context.	Apply access filter before context assembly and test with instrumented mock model to prove denied chunks were never sent.

Risk	Impact	Mitigation / owner
Schema target-state mistaken for deployable migration	Unreviewed 158-table DDL may fail or overprovision.	Stage migrations by bounded domain and dependencies, validate against a disposable DB, inspect indexes/constraints and keep R2 DDL separate.
Financial methods not specified precisely enough	Outputs vary or cannot be reproduced.	Create ADRs for market calendar/timezone, corporate-action adjustments, DCF conventions, risk methods, backtest fill/cost and bias policy.
Long-running job partial failure	Duplicate vectors, snapshots, reports or inconsistent status.	Durable run/attempt records, idempotency keys, bounded retry, reconciliation jobs and automated chaos tests.
License / redistribution restrictions	Production use may violate provider terms.	Keep provider source/entitlement metadata; legal/product signoff for storage/display/export/redistribution.

19.3 Open decisions requiring explicit confirmation

- Authentication implementation: choose built-in identity store vs configured identity provider; confirm password reset/MFA policy and session/cookie model.
- Market data: choose specific provider(s), credential delivery, licensing, supported universes, adjusted/unadjusted bars, corporate-action policy and historical coverage.
- Embedding model and index schema: approve provider/model/version/dimension, chunk size/overlap, similarity/reranking policy and re-index behavior.
- Compute conventions: benchmark, exchange calendars/timezones, base currencies, return annualization, DCF WACC/terminal-value conventions, VaR/CVaR variants and backtest fill/cost assumptions.
- Hosting/deployment: choose target environment, secret manager, object storage, TLS termination, worker scaling, backup cadence and numerical RTO/RPO.
- User/workspace model: confirm multi-tenant expectations, roles, sharing grants and retention/export/deletion policy.
- Release 2: establish separate acceptance criteria, broker sandbox, legal/compliance and incident-control review before any order API is built or enabled.

20. Definition of Done and release acceptance

20.1 Story-level Definition of Done

- Jira story maps to SRS FR/NFR references and has testable acceptance criteria; dependencies and release label are explicit.
- Implementation follows module ownership and API/schema contracts; no hidden cross-domain database writes without approved reason.
- Automated unit/API/integration tests appropriate to the story pass; failure cases, authorization, data-quality, retries and resource limits are covered.
- Code review completed; lint/static/security checks considered and findings either fixed or documented with accepted owner/rationale.
- Database changes have forward migration, validation and appropriate recovery/rollback strategy; data provenance and access constraints persist.
- Run/log/citation/snapshot metadata are present where applicable; sensitive values are redacted.
- Documentation, OpenAPI examples, ADRs, test IDs and traceability matrix updated; PR, CI result and evidence linked in Jira.
- For partial implementation of a grouped story, create subtasks and leave parent not Done until all original acceptance criteria pass.

20.2 Cross-system release gate

Gate	Pass condition
Functional scope	All stories advertised as Done satisfy their full original Jira acceptance checks; deferred or partial scope is marked accurately.
Critical security	No unresolved critical authorization, cross-tenant disclosure, secret exposure, data corruption or exploitable injection issue.
Data correctness	Provider and fixture paths are labelled; snapshots/assumptions/config versions persist; numerical reference tests and ingestion idempotency pass.

Gate	Pass condition
RAG evidence	Private retrieval permissions enforced before model context; citations resolve to source and index lifecycle is reconciled.
Operations	Readiness checks, structured/redacted logs, backup/restore evidence, bounded retries, health monitoring and deployment/rollback runbook exist.
Release boundary	Release 1 build contains no enabled order-execution capability; optional trading schema is not applied; R2-only aliases remain marked future.
Evidence	CI/test/performance/security reports and demos are attached to the release record; actual outcomes are observed, not inferred from documentation.

Final technical lead recommendation

Use this document as the design baseline and the 72-entry schedule as the planning baseline. The first integration risk is contract drift between modules, so freeze API/domain DTOs, identifiers, snapshot semantics and permission policy early. The most important quality invariant is provenance plus authorization: every sensitive datum and every generated financial result must be explainable, scoped to its owner, and reproducible from recorded inputs.

Appendix A. Complete story-to-sprint allocation

Every alias below comes from the supplied four-sprint schedule and complete Jira catalog. This appendix is a completeness index; the sprint tables above contain the implementation/acceptance focus. Jira will assign actual issue keys, so preserve the FS aliases in a custom field, label or description.

Sprint 1 story index

Jira alias	Source story title	SP	Primary component
FS-11-01	Create reproducible repository and environment setup	2	Repo/DevEx
FS-11-02	Run CI quality gates on every pull request	3	CI/CD
FS-11-03	Build and deploy a versioned release artifact	3	CI/CD
FS-11-04	Apply TDD and maintain unit/integration/E2E suites	5	QA platform
FS-11-05	Measure line/branch coverage and test quality	2	QA metrics
FS-11-06	Run security validation and dependency checks	3	DevSecOps
FS-11-07	Document architecture, APIs, ADRs and compatibility rules	2	Architecture docs
FS-11-08	Maintain SRS addendum and requirement traceability matrix	3	Requirements
FS-11-09	Run planning, daily review, demo and retrospective	2	Delivery
FS-11-10	Manage baselines, bugs, versions, SBOM and release notes	2	Release engineering
FS-11-11	Document usability, AI use, licenses and data rights	2	Governance
FS-11-12	Add health checks, structured logs, retry and recovery runbook	3	SRE / ops
FS-01-01	Authenticate and manage sessions	3	Identity
FS-01-02	Enforce roles and protected routes	3	Identity

Jira alias	Source story title	SP	Primary component
FS-01-03	Protect accounts and audit authentication	3	Identity/security
FS-01-04	Separate read-only and trading permissions	2	Identity/security
FS-01-05	Meet identity security and accessibility requirements	3	Identity/security
FS-08-01	Configure Ollama, model selection and inference parameters	3	AI configuration
FS-08-05	Secure and validate configuration changes	3	Configuration
FS-09-03	Manage API credentials, privacy, export, deletion and audit	3	Privacy/security
FS-09-05	Enforce privacy and safe account administration	3	Privacy/security
FS-07-05	Secure execution logs and retain reliable run history	3	Observability
FS-02-08	Keep dashboard responsive, isolated and freshness-aware	2	Frontend / quality

Sprint 2 story index

Jira alias	Source story title	SP	Primary component
FS-05-01	Register and configure data sources securely	3	Market data
FS-05-02	Query structured datasets safely	3	Data Explorer
FS-05-03	Ingest market prices, instruments, fundamentals and events	5	Market data
FS-05-04	Import, preview, clean, transform and combine datasets	5	Data Explorer
FS-05-05	Backfill and refresh data with quality and lineage	5	Market data
FS-05-06	Enforce least privilege, query safety and source isolation	3	Data security
FS-04-01	Upload, validate and extract documents	3	Research ingestion
FS-04-02	Chunk, embed, index and track processing	5	RAG indexing
FS-04-03	Search, preview, filter and manage research documents	3	Research UI/API
FS-04-04	Handle duplicates and recover processing failures	3	RAG jobs
FS-04-06	Protect file processing, index consistency and document lifecycle	3	RAG security
FS-08-02	Configure embeddings, chunking and retrieval	3	RAG configuration
FS-08-03	Manage market providers, data coverage and analytics defaults	3	Provider configuration
FS-02-01	Show dashboard health and activity	3	Dashboard/ops
FS-02-02	Show market overview, sessions, movers and heatmaps	3	Market dashboard
FS-02-03	Explore interactive historical price charts	5	Frontend/market data

Jira alias	Source story title	SP	Primary component
FS-09-01	Manage profile, security, interface and notification preferences	2	Account/workspace

Sprint 3 story index

Jira alias	Source story title	SP	Primary component
FS-03-01	Create and manage AI conversations	3	AI analyst
FS-03-02	Stream, stop and regenerate answers	3	AI analyst
FS-03-03	Answer with authorized retrieval and precise citations	5	RAG / AI security
FS-03-04	Answer financial questions using deterministic tools	5	AI/finance services
FS-03-05	Create natural-language screening requests	3	AI/screening
FS-03-06	Search, export, save and invoke approved workflows	3	AI / saved research
FS-03-07	Make AI behavior safe, traceable and understandable	3	AI safety
FS-06-01	Create, configure, validate, test and version workflows	5	Workflow studio
FS-06-02	Run, schedule and monitor workflows with approvals	5	Workflow / jobs
FS-06-03	Build and save stock screens	3	Screening
FS-06-04	Define versioned strategies, variants and readiness checks	5	Strategy engine
FS-06-05	Generate explainable signals and alerts safely	3	Signals / alerting
FS-06-06	Make workflow execution secure, bounded and reproducible	3	Workflow security
FS-02-05	Manage watchlists	3	Workspace / dashboard
FS-02-06	View portfolio performance, allocation and risk	5	Portfolio analytics
FS-02-07	View news, events, alerts and custom dashboard layout	3	Dashboard / saved UI
FS-09-02	Manage workspaces, members, roles and sharing	3	Workspace/security

Sprint 4 story index

Jira alias	Source story title	SP	Primary component
FS-07-01	Inspect execution history and troubleshoot jobs	3	Jobs / operations
FS-07-02	Configure backtests and calculate performance metrics	5	Backtesting
FS-07-03	Analyze, compare, validate and export backtests	5	Backtesting / reporting
FS-07-04	Implement paper trading and broker execution controls	8	Broker/trading
FS-10-01	Calculate DCF valuation with versioned assumptions	5	Valuation

Jira alias	Source story title	SP	Primary component
FS-10-02	Calculate portfolio VaR/CVaR and drawdown	5	Risk analytics
FS-10-03	Run Monte Carlo and stress scenarios	5	Risk analytics
FS-10-04	Manage portfolios, transactions and snapshots	5	Portfolio domain
FS-10-05	Calculate returns, attribution and allocation	5	Portfolio analytics
FS-10-06	Generate source-backed PDF/CSV reports	3	Reporting
FS-10-07	Share, reopen and version saved research	3	Reporting / saved research
FS-09-04	Save dashboards, watchlists, screens, strategies, reports and preferences	3	Saved research
FS-04-05	Enrich financial metadata and track document versions	5	Research lifecycle
FS-02-04	Compare instruments and fundamentals	3	Market analytics
FS-08-04	Keep broker modes and trading safety controls gated	2	Broker configuration

Appendix B. Source references and traceability map

Reference artifact	Design sections that consume it	Traceability rules
FinSight Software Requirements Specification (SRS), 49 pages	System scope/architecture; P1-P9 functional requirements; NFR-G-01..NFR-G-38; page relationships and key workflows.	Keep FR/NFR IDs in story/test descriptions. If a summary omits a requirement, defer only by a recorded scope decision.
FinSight AI Complete Jira Scrum Backlog, 12 pages	11 epics E1-E11, 72 grouped story/task catalog, story titles/points/priorities/acceptance criteria.	Use the actual Jira key plus source alias FS-xx-xx; map alias to epic and SRS range.
FinSight AI Full Project Four-Sprint Plan, 12 pages	All four sprint assignments; story load counts; dependencies and release labels.	Use four-sprint allocation only. Do not use the superseded seven-day plan.
FinSight AI Database Schema and ERD Reference, 81 pages	PostgreSQL/pgvector, domain table counts, identity, market, RAG, portfolio, risk/valuation, workflow/backtest and optional trading relationships.	SQL/schema files remain canonical for exact names, types, PK/FK, indexes and constraints; verify migrations live.
FinSight AI Sprint 1 Test Plan, 33 pages	54 Sprint 1 tests; story-to-test mapping; CI/security/system test rules.	Reuse IDs where applicable, expand tests for Sprints 2-4, and keep actual result/status blank until execution.

Key source-derived conventions

- The schema reference enumerates 173 table definitions in total: 158 core plus 15 optional trading tables. The table catalogue validates names/FK targets statically, but it does not claim that a live PostgreSQL migration was successfully executed.
- The full four-sprint schedule allocates all 72 story aliases exactly once; loads are 23/17/17/15 entries with 63/60/63/65 original points.
- The source release labels FS-07-04 and FS-08-04 remain Future - Release 2 only.
- The source SRS performance values are targets tied to reference conditions and must be measured on documented hardware, dataset, model and workload.